

AI 使用说明情况（第二次挑战）

学号 0102603133

一、使用的 AI 模型

工具：CodeBuddy（代理式编码助手），由对话式大模型驱动。使用方式与第三次挑战一致：沙箱内自主执行，通过 SSH 在 AutoDL 实例上跑命令，我负责目标设定与验收。全程未使用其它 AI 工具。

二、使用的 AI 提示词

初始提示词（大意）：附三份挑战 PDF、HW3-scaffold.zip 与实例 SSH 信息，要求先对照第二关 PDF 做事实核查、再落地完成两次挑战；特别强调参数必须与文档一致（32×2 条、并发 8、temperature=0、max_new_tokens=16、ignore_eos=true、sampling_seed=2026、预热不计入结果）。**过程中的追加指令：**「需要更换镜像吗」「版本不是太低了吗」「换个高一点的」「我用的是 autodl 可以在控制台直接更换 torch 的版本」「？」——分别触发 pip 源测速、torch 升级方案与进度确认。

三、AI 如何帮助你学习

测量方法：帮我设计并实现 measure_prefix_cache.py：并发控制在 8、每轮先等在飞结束再 POST /flush_cache 并校验、共享前缀组先发预热请求把公共前缀载入 RadixCache；从流式响应的 meta_info 里解析 cached_tokens / prompt_tokens / completion_tokens / ttft / tpot，逐请求落 CSV 再汇总。**指标理解：**把「prefill 削减 89.2%」与「TPOT 只小幅变化」用 TTFT/TPOT 的定义拆开解释清楚，并从因果自注意力公式推导了前缀 K/V 可复用的原因。**源码阅读：**任务二的流程图不是我凭感觉画的——AI 带我沿 TokenizerManager → Scheduler → PrefillAdder → RadixCache → TpModelWorker → DetokenizerManager 的主线，在本机安装的 0.5.14 源码里逐个函数 grep 定位并读关键片段，流程图上的每个「文件：行号」都实测核对过。

四、在运用 AI 时是否被误导

有。（1）**API 假设过时：**AI 假设 /flush_cache 返回 JSON，实际 0.5.14 返回纯文本「Cache flushed」（还需要轮询到缓存真正清空），第一版校验一直失败。（2）**凭记忆的行号不可靠：**AI 最初按记忆描述的函数位置与实际安装版本有出入，后来全部改为在源码里 grep 重新定位——这条教训是：读源码必须以本机版本为准。（3）**升级建议的陷阱：**AI 一开始建议换 AutoDL 基础镜像升级 torch，会清空系统盘；权衡后改为在现有环境内 pip 升级 torch 2.11.0+cu130。

五、边界声明

目标一全部数字来自真实运行产物（results/target1/ 下逐请求 CSV 与汇总 JSON 可逐一对齐）；任务二流程图与说明中的函数定位均在本机 sglang 0.5.14 源码中实测。按课程要求，作业感受第（2）（3）问由本人独立撰写，未使用 AI。